B02 | 请求怎样进入 Python 函数?

本章看服务端接手的顺序：**收到请求 → 匹配入口 → 解析并检查输入 → 函数处理当前状态 → 检查公开结果 → 形成响应。** 重点不是默写整份服务，而是知道失败时已经走到哪里。

请求和函数之间，谁在牵线?

当前问题	关系或短用法	记到什么程度
有了 `app = FastAPI()`，服务就监听了吗?	它建立应用对象；Uvicorn 加载应用并监听，FastAPI 负责接口处理，业务函数决定工单怎样变化。	**应用定义、实际运行、业务处理分开。** 导入框架不等于端口已经可用。
请求没写函数名，怎么找到代码?	`@app.post("/tickets", status_code=201)` 登记 POST、路径和紧随其后的函数。加载时登记，匹配请求后才调用。	认熟 method + path 选择入口；函数叫 `create_ticket` 不会自动形成 HTTP 入口。
JSON 怎样成为函数可用的值?	`payload: CreateTicket` 让框架按模型解析、检查 body，再把对象交给参数；`payload.title` 取属性。	知道检查由框架执行；普通 Python 类型注解本身不会自动校验所有函数调用。
path、query、body 中的值从哪取?	本例路径中的 `{ticket_id}` 接编号；`include_history: bool = False` 接 query，转换并提供默认值；创建内容进 body model。	能指出"处理谁、读取选项、提交内容"的位置。`response: Response` 由框架提供，不是客户端填入的字段。
`print` 和 `return` 都能回应吗?	`return record` 把值交给框架组织响应；`print(record)` 写到服务输出。	**返回给代码与显示到终端分开。** 服务打印了记录，不表示客户端已收到它。

`mini_api:app` 读成"`mini_api` 模块中的 `app` 对象"，不是带冒号的文件名；完整启动参数查用。这个小例也不替换完整配套 `app.py`。

schema 描述允许的数据形状，不是某次具体值。`Field(ge=1, le=3)` 表示至少 1、至多 3，`Literal[...]` 限定取值；需要改字段时查模型。FastAPI 据 route、模型等声明生成 `/openapi.json` 和 `/docs`，**文档反映声明，不替代实际请求验证**。

回看：「先让一条请求找到一段代码」「FastAPI 和启动服务的工具是什么关系？」「接口文档怎么知道要填写哪些字段？」「URL 里的编号和选项怎样进入函数？」。

这次拒绝发生在哪里，之前有没有改过状态?

当前问题	关系或短用法	记到什么程度
合法 JSON 就能进入创建函数吗?	`"priority":"urgent"` 可解析，却不满足输入模型；本例 422 在创建函数前发生，没有新增记录或消耗编号。	区分 JSON 解析和输入校验。默认配置也可能把数字文字转换为整数，不概括成“所有字符串都拒绝”。
得到 404，函数一定没运行吗?	路径未匹配可以 404；匹配后查不到工单，也可以由函数返回 404。`raise HTTPException(...)` 把失败交给框架形成回应。	看目标和错误证据；只在返回字典里写“错误”不会自动改变状态码，抛 HTTPException 也不等于杀掉服务。
版本号类型正确，为什么还是拒绝修改?	模型查形式和范围；函数比较 `expected_version` 与当前记录，不同返回 409，相同才修改并增加版本。	**输入合法不等于当前业务条件成立。** 示例内存中的先比较再写，没有并发原子保证。
内部结果为什么不全发出去?	`response_model=TicketPublic` 检查公开形状、过滤未声明字段，内部存储的字段没有因此删除。	区分内部记录和公开表示；输出过滤不替代权限判断。
服务自己返回错字段，为什么不是 422?	服务违背自己的输出约定，正常运行默认形成 500；此前已发生的写入不会自动撤销。	**错误归属与业务效果分别判断。** 收到失败，不等于创建从未发生。

读取历史时怎样复制字典和列表、选择是否返回历史、是否省略 None，都在原章查用。这里要保留的关系是：改变公开表示不应意外改坏内部记录；浅拷贝也不等于复制全部嵌套对象。

回看：「为什么先检查输入，再运行创建函数?」「得到 404，说明函数没有运行吗?」「输入都合法了，为什么还可能不能修改?」「函数 return 的内容，为什么不全部交给客户端?」「服务自己返回错了字段，为什么不是 422?」。

测试、状态与配置，分别看哪一段?

当前问题	关系或短用法	记到什么程度
直接调函数，与测试一次接口差在哪?	TestClient 在测试进程内把请求交给应用，经过路由、解析和校验，不需要真实监听端口。	逐渐用熟 `client.post(..., json=...)`、`status_code`、`json()`；**接口测试通过，不证明端口在监听。**
拒绝输入，只断言 422 够吗?	还要核对关心的业务效果：本例工单集合和下一编号没有被错误输入改变。`create_app(...)` 可让各测试有独立应用。	根据要保护的结果选断言，不凑数量；测试初始化不能套到正在使用的数据上。
下一次请求还看得到记录，是否已经保存到磁盘?	`app.state.tickets` 在当前应用内存中跨请求保留；服务重启、重新建立应用后回到空状态。	**跨请求保留不等于持久保存。** 后续由 C 册连接数据库。
每次提交的标题，与供应商地址都放 body 吗?	标题来自本次请求；`Settings()` 读取启动环境等配置。另一终端后来改变量，不会自动更新已运行应用。	分清来源和读取时机；简单读取可用 `os.getenv`，多项配置的类型和默认值可集中处理。

少用选项按问题查：TestClient 默认抛出服务端异常，想观察外部 500 时才用 `raise_server_exceptions=False`；Settings 字段别名和 `SecretStr` 的完整写法也不用默写。`SecretStr` 仅帮助常规显示时掩码，手动取值、打印仍能泄露。本例无需真实凭据。

回看：「怎样检查整个接口，而不只检查函数?」「哪些值属于请求，哪些值属于运行配置?」。

服务端检查了自己的处理，但客户端面对的是**实际收到的回应**，不能假定它必然符合意图。B03 从这个边界继续。